

EAP: Architecture Specification and Prototype

With the growth of the Internet, misinformation has become and continues to be a global problem, often leading people to make poor decisions due to a lack of factual knowledge about events. Our goal is to create a platform where users can share news, while fact-checkers and administrators ensure that content is properly sourced, and where discussion is fostered among people from diverse backgrounds, perspectives and beliefs.

Our mission is to promote No-Mis-Information and No-Miss-Information.

A7: Web Resources Specification

This artifact documents the architecture of our web application. This artifact shows the available resources, their properties and responses. The specification follows the OpenAPI standard using the YAML format.

In here, we also present the documentation for NoMisHub, including the implemented CRUD operations of the resources present in the vertical prototype.

1. Overview

This section provides an overview of the complete web application to be implemented, identifying and describing its main modules. The web resources included in the vertical prototype are further detailed in each module's individual documentation within the OpenAPI specification.

Module	Description
M01: Authentication and Profile View and Management	Web resources associated with authentication and user profile management. Includes the following system functionalities: login/logout, registration, password recovery, viewing and editing personal profile information and viewing other users' profiles.
M02: News	Web resources associated with news items. Includes the following system features: news listing and search, news detail viewing and editing, and news deletion.
M03: Comments and Votes	Web resources associated with comments and votes. Includes the following system functionalities: adding comments and votes, listing comments, deleting comments and votes and editing comments.
M04: Admin/Moderator Management	Web resources associated with user and content management, specifically: deleting or promoting user accounts, viewing and changing user information, viewing system access details for each user and managing user content.
M05: Static Pages	Web resources associated with static content: about us, help and frequently asked questions.

2. Permissions

This segment describes the permissions used in the last section (modules) to settle the conditions of access to resources.

Identifier	Name	Description
PUB	Public	Users without privileges
USR	User	Authenticated users
OWN	Owner	Users who own a specific resource (e.g., their profile, news, or comments)
ADM	Administrator	NoMisHub administrators
MOD	Moderator	NoMisHub moderators

3. OpenAPI Specification

The OpenAPI specification can also be seen here:

```
openapi: 3.0.0

info:
  version: '1.0'
```

```
title: 'NoMisHub Web API'
description: 'Web Resources Specifiction (A7) for NoMisHub'

servers:
- url: http://db.fe.up.pt
  description: Production server

tags:
- name: 'M01: Authentication and Profile View and Management'
- name: 'M02: News'
- name: 'M03: Comments and Votes'
- name: 'M04: Admin/Moderator Management'
- name: 'M05: Static Pages'

paths:

/login:
  get:
    operationId: R101
    summary: 'R101: Login Form'
    description: 'Provide login form. Access: PUB'
    tags:
      - 'M01: Authentication and Profile View and Management'
    responses:
      '200':
        description: 'Ok. Show login UI'
  post:
    operationId: R102
    summary: 'R102: Login Action'
    description: 'Process the login form. Access: PUB'
    tags:
      - 'M01: Authentication and Profile View and Management'

    requestBody:
      required: true
      content:
        application/x-www-form-urlencoded:
          schema:
            type: object
            properties:
              username: # <!-- form field name
                type: string
              password: # <!-- form field name
                type: string
            required:
              - username
              - password

    responses:
      '302':
        description: 'Redirect after processing the login credentials.'
        headers:
          Location:
            schema:
              type: string
            examples:
              302Success:
                description: 'Successful authentication. Redirect to user profile.'
                value: '/users/{id}'
              302Error:
                description: 'Failed authentication. Redirect to login form.'
                value: '/login'

/logout:

  post:
    operationId: R103
```

```
summary: 'R103: Logout Action'
description: 'Logout authenticated used. Access: USR'
tags:
  - 'M01: Authentication and Profile View and Management'
responses:
  '302':
    description: 'Redirect after processing logout.'
    headers:
      Location:
        schema:
          type: string
        examples:
          302Success:
            description: 'Successful logout. Redirect to main page.'
            value: '/'

/register:
  get:
    operationId: R104
    summary: 'R104: Register Form'
    description: 'Provide registration form. Access: PUB'
    tags:
      - 'M01: Authentication and Profile View and Management'
    responses:
      '200':
        description: 'Ok. Show sign-up UI'

  post:
    operationId: R105
    summary: 'R105: Register Action'
    description: 'Processes the registration form. Access: PUB'
    tags:
      - 'M01: Authentication and Profile View and Management'

  requestBody:
    required: true
    content:
      application/x-www-form-urlencoded:
        schema:
          type: object
          properties:
            username:
              type: string
            name:
              type: string
            email:
              type: string
            password:
              type: string
          required:
            - username
            - name
            - email
            - password

  responses:
    '302':
      description: 'Redirect after processing the new user information.'
      headers:
        Location:
          schema:
            type: string
          examples:
            302Success:
              description: 'Successful registration. Redirect to user profile.'
              value: '/users/{id}'
            302Failure:
```

```
description: 'Failed registration. Redirect to login form.'
value: '/register'

/users/{user}:
  get:
    operationId: R106
    summary: 'R106: View user profile'
    description: 'Show the individual user profile. Access: PUB'
    tags:
      - 'M01: Authentication and Profile View and Management'

    parameters:
      - in: path
        name: user
        schema:
          type: object
        required: true

    responses:
      '200':
        description: 'Ok. Show view profile UI.'
      '404':
        description: 'Not Found. User not found.'

/user/edit/{user}:
  get:
    operationId: R107
    summary: 'R107: View user information edit form'
    description: 'Show the user interface for profile edition. Access: OWN, ADM'
    tags:
      - 'M01: Authentication and Profile View and Management'
      - 'M04: Admin/Moderator Management'

    parameters:
      - in: path
        name: user
        schema:
          type: object
        required: true

    responses:
      '200':
        description: 'Ok. Show user profile edition UI.'
      '404':
        description: 'Not Found. User not found.'
      '403':
        description: 'Forbidden. User not allowed.'

  post:
    operationId: R108
    summary: 'R108: Process the profile edit form'
    description: 'Process the profile editing form. Access: OWN, ADM'
    tags:
      - 'M01: Authentication and Profile View and Management'
      - 'M04: Admin/Moderator Management'

    requestBody:
      required: true
      content:
        application/x-www-form-urlencoded:
          schema:
            type: object
            properties:
              username:
                type: string
              name:
                type: string
```

```
      image:
        type: string
        format: binary
      email:
        type: string
      current_password:
        type: string
      new_password:
        type: string
      required:
        - current_password

responses:
  '302':
    description: 'Redirect after processing the new profile information.'
    headers:
      Location:
        schema:
          type: string
    examples:
      302Success:
        description: 'Successful editing. Redirect to user profile.'
        value: '/users/{id}'
      302Failure:
        description: 'Failed editing. Redirect to edit profile form.'
        value: '/users/edit/{id}'
  '404':
    description: 'Not Found. User not found.'
  '403':
    description: 'Forbidden. User not allowed.'
```

/user/delete/{user}:

get:

operationId: R109

summary: 'R109: View user information deletion form'

description: 'Show the user interface for profile deletion. Access: OWN, ADM'

tags:

- 'M01: Authentication and Profile View and Management'
- 'M04: Admin/Moderator Management'

parameters:

- in: path
- name: user
- schema:
 type: object
 required: true

responses:

'200':

description: 'Ok. Show user profile deletion UI.'

'404':

description: 'Not Found. User not found.'

'403':

description: 'Forbidden. User not allowed.'

post:

operationId: R110

summary: 'R110: Delete the account.'

description: 'Delete th account. Access: OWN, ADM'

tags:

- 'M01: Authentication and Profile View and Management'
- 'M04: Admin/Moderator Management'

requestBody:

required: true

content:

application/x-www-form-urlencoded:

```

    schema:
      type: object
      properties:
        id:
          type: integer
      required:
        - id

  responses:
    '302':
      description: 'Redirect after deleting the account.'
      headers:
        Location:
          schema:
            type: string
          examples:
            302Success:
              description: 'Successful deletion. Redirect to main page.'
              value: '/news'
    '404':
      description: 'Not Found. User not found.'
    '403':
      description: 'Forbidden. User not allowed.'
```

/news:

get:

operationId: R201

summary: 'R201: View all news'

description: 'Show all news. Access: PUB'

tags:

- 'M02: News'

responses:

'200':

description: 'Ok. Show view all news UI.'

/news/{news}:

get:

operationId: R202

summary: 'R202: View news page'

description: 'Show the individual news page. Access: PUB'

tags:

- 'M02: News'

parameters:

- in: path
 name: news
 schema:
 type: object
 required: true

responses:

'200':

description: 'Ok. Show view news UI.'

'404':

description: 'Not Found. News not found.'

/news/edit/{news}:

get:

operationId: R203

summary: 'R203: View news edit form'

description: 'Show the user interface for news edition. Access: OWN, ADM'

tags:

- 'M02: News'
- 'M04: Admin/Moderator Management'

parameters:


```
- in: path
  name: news
  schema:
    type: object
  required: true

responses:
  '200':
    description: 'Ok. Show user news edition UI.'
  '404':
    description: 'Not Found. News not found.'
  '403':
    description: 'Forbidden. User not allowed.'

post:
  operationId: R204
  summary: 'R204: Process the news edit form'
  description: 'Process the news editing form. Access: OWN, ADM'
  tags:
    - 'M02: News'
    - 'M04: Admin/Moderator Management'

requestBody:
  required: true
  content:
    application/x-www-form-urlencoded:
      schema:
        type: object
        properties:
          image:
            type: string
            format: binary
          title:
            type: string
          content:
            type: string

responses:
  '302':
    description: 'Redirect after processing the new profile information.'
    headers:
      Location:
        schema:
          type: string
        examples:
          302Success:
            description: 'Successful editing. Redirect to news page.'
            value: '/news/{id}'
          302Failure:
            description: 'Failed editing. Redirect to edit news page.'
            value: '/news/edit/{id}'
  '404':
    description: 'Not Found. News not found.'
  '403':
    description: 'Forbidden. User not allowed.'

/news/delete/{news}:
get:
  operationId: R205
  summary: 'R205: View news deletion form'
  description: 'Show the user interface for news deletion. Access: OWN, ADM'
  tags:
    - 'M02: News'
    - 'M04: Admin/Moderator Management'

parameters:
  - in: path
```

```

    name: news
    schema:
      type: object
    required: true

responses:
  '200':
    description: 'Ok. Show news deletion UI.'
  '404':
    description: 'Not Found. News not found.'
  '403':
    description: 'Forbidden. User not allowed.'

post:
  operationId: R206
  summary: 'R206: Delete the news piece.'
  description: 'Delete the news piece. Access: OWN, ADM'
  tags:
    - 'M02: News'
    - 'M04: Admin/Moderator Management'

  requestBody:
    required: true
    content:
      application/x-www-form-urlencoded:
        schema:
          type: object
          properties:
            id:
              type: integer
          required:
            - id

  responses:
    '302':
      description: 'Redirect after deleting the news piece.'
      headers:
        Location:
          schema:
            type: string
          examples:
            302Success:
              description: 'Successful deletion. Redirect to main page.'
              value: '/news'
    '404':
      description: 'Not Found. News not found.'
    '403':
      description: 'Forbidden. User not allowed.'

/news/create:
  get:
    operationId: R207
    summary: 'R207: View news creation form'
    description: 'Show the user interface for news creation. Access: USR'
    tags:
      - 'M02: News'

    responses:
      '200':
        description: 'Ok. Show user news creation UI.'

  post:
    operationId: R208
    summary: 'R208: Process the news creation form'
    description: 'Process the news creation form. Access: USR'
    tags:
      - 'M02: News'
```



```
requestBody:
  required: true
  content:
    application/x-www-form-urlencoded:
      schema:
        type: object
        properties:
          image:
            type: string
            format: binary
          title:
            type: string
          content:
            type: string
        required:
          - title
          - content

responses:
  '302':
    description: 'Redirect after processing the new news information.'
    headers:
      Location:
        schema:
          type: string
    examples:
      302Success:
        description: 'Successful creation. Redirect to news page.'
        value: '/news/{id}'
      302Failure:
        description: 'Failed creation. Redirect to news creation page.'
        value: '/news/create'
```

The OpenAPI specification can also be seen here: [<OpenAPI>](#)

A8: Vertical prototype

The Vertical Prototype includes the implementation of the high priority features marked as necessary in the project requirements and in the [Prototype] marked User Stories. This prototype allowed us to get better acquainted with the used technologies.

The implementation is based on the LBAW Framework and includes work on all layers of the architecture of the solution to implement: user interface, business logic and data access, with features ranging from the showing of information (users and news) to the management (creation, editing and deletion) of the core components of the system (Users and News) and the display of errors.

1. Implemented Features

1.1. Implemented User Stories

User Story reference	Name	Priority	Description	Implementation Status
US01	Top News	Prototype	As a user, I want to be able to see which are the current top news, so that I can be on par with the most important news of the time.	Implemented
US02	News Item	Prototype	As a user, I want to be able to see more information about a news item and its comments, so that I can interact with it and get to know more about it.	Implemented

User Story reference	Name	Priority	Description	Implementation Status
US03	Exact Match Search	Prototype	As a user, I want to have access to be able to perform an exact match search, so that I can find a news article or comment by its title/content.	Implemented
US04	Full-text Search	Prototype	As a user, I want to have access to full-text search, so that I can find the news piece or comment I'm searching for without knowing its title/content.	Implemented
US15	Login	Prototype	As a guest, I want to be able to log in into my account, so that I can have access to functionalities related to my account.	Implemented
US16	Registration	Prototype	As a guest, I want to be able to register a new account, so that I can create an account and get access to more functionalities.	Implemented
US18	Logout	Prototype	As an authenticated user, I want to be able to log out my account, so that I can end my session.	Implemented
US19	View Profile	Prototype	As an authenticated user, I want to be able to view my profile, so that I can check my information.	Implemented
US20	Manage Profile	Prototype	As an authenticated user, I want to be able to edit my profile, so that I can update of my personal information.	Implemented
US21	User News Feed	Prototype	As an authenticated user, I want to be able to view a personalized news feed based on my follows, so that I can quickly find news that are more relevant for my interests.	Partial
US22	News Item Details	Prototype	As an authenticated user, I want to be able to see details about a particular news item, so that I can quickly conclude whether it's recent news, how appreciated it is among others.	Implemented
US23	Publish News Item	Prototype	As an authenticated user, I want to be able to publish a news item, so that I can share any news I have with other users.	Implemented
US24	View Others' Profiles	Prototype	As an authenticated user, I want to be able to see other users' profiles so that I can see more information about them like reputation and posted articles and comments.	Implemented
US25	Manage News Item	Prototype	As an authenticated user, I want to be able to edit or delete news items that I posted, so that I can update them with accurate information or delete them needed.	Implemented

User Story reference	Name	Priority	Description	Implementation Status
US40	Manage User Accounts	Prototype	As an administrator, I want to have tools that would allow me to manage other users' accounts like editing, banning and deleting, so that I can take appropriate action if a user infringes the rules.	Implemented

1.2. Implemented Web Resources

Module M01: Authentication and Profile View and Management

Web Resource Reference	URL
R101: Login Form	GET /login
R102: Login Action	POST /login
R103: Logout Action	POST /logout
R104: Register Form	GET /register
R105: Register Action	POST /register
R106: User Profile	GET /user/{user}
R107: User Edit Form	GET /user/edit/{user}
R108: User Edit Action	POST /user/edit/{user}
R109: User Delete Form	GET /user/delete/{user}
R110: User Delete Action	POST /user/delete/{user}

Module M02: News

Web Resource Reference	URL
R201: All News Page	GET /news
R202: News Page	GET /news/{news}
R203: Edit News Form	GET /news/edit/{news}
R204: Edit News Action	POST /news/edit/{news}
R205: Delete News Form	GET /news/delete/{news}
R206: Delete News Action	POST /news/delete/{news}
R207: Create News Form	GET /news/create
R208: Create News Action	POST /news/create

2. Prototype

The Docker image of the prototype is available on GitLab and can be run with:

```
docker run -d --name lbaw2511 -p 8001:80 gitlab.up.pt:5050/lbaw/lbaw2526/lbaw2511
```

The application will be available at <http://localhost:8001>

Credentials (username/password):

- admin user: admin/testing123
- regular user: regUser/testing123

The tag that represents the state of the repository at the point of this delivery can be found here: [<EAP>](#)

Revision history

Changes made to the first submission:

1.

GROUP2511, 24/11/2025

- Ana Catarina Barbosa Patrício, up202107383@fe.up.pt
- António Pedro Alves Pais, up202305444@fe.up.pt
- Hugo Ribeiro Alves, up202305395@fe.up.pt
- Tomás Pereira da Silva, up202307796@fe.up.pt

Editor: Tomás Pereira da Silva

Comments